comπle Editor: JavaScript Implementation Guide

A detailed breakdown of Phase 1 (The Shell) and Phase 3 (The Bridge) using pure JavaScript.

Step 4: Configure the Secure Bridge (`preload/index.js`)

The preload script acts as the secure middleman between the backend (Node.js) and the frontend (React).

```
import { contextBridge, ipcRenderer } from 'electron'
import { electronAPI } from '@electron-toolkit/preload'

const api = {
  getFileContents: (filePath) => ipcRenderer.invoke('get-file-contents', filePath),
  sendAIPrompt: (prompt) => ipcRenderer.invoke('send-ai-prompt', prompt),
  onAIStream: (callback) => {
      ipcRenderer.removeAllListeners('ai-stream-chunk');
      ipcRenderer.on('ai-stream-chunk', (_event, chunk) => callback(chunk));
  }
}

if (process.contextIsolated) {
  try {
    contextBridge.exposeInMainWorld('electron', electronAPI)
    contextBridge.exposeInMainWorld('api', api)
  } catch (error) {
    console.error(error)
  }
} else {
  window.electron = electronAPI
```

```
  window.api = api
}
```

How it works:

- `contextBridge` is a security feature. It ensures React cannot directly access native Node.js modules like `fs` (file system), which prevents malicious code from hijacking the user's computer.
- We define an `api` object containing functions. These functions use `ipcRenderer.invoke` to send messages across the IPC (Inter-Process Communication) bridge to the backend.
- We use `contextBridge.exposeInMainWorld('api', api)` to attach these secure functions to the global `window` object in React, so you can call them as `window.api.getFileContents()`.

Step 5: Configure the Backend (`main/index.js`)

The main process runs in Node.js. It manages the application lifecycle and listens for messages from the preload script.

```
// --- IPC HANDLERS (The Backend Logic) ---

ipcMain.handle('get-file-contents', async (event, filePath) => {
    console.log(`Backend received request for: ${filePath}`);
    // In a real app, you would use: return await fs.promises.readFile(filePath,
    return `Simulated contents of ${filePath} loaded securely!`;
});

ipcMain.handle('send-ai-prompt', async (event, prompt) => {
    console.log(`Sending to AI: ${prompt}`);

    // Simulate streaming chunks back to the frontend
    setTimeout(() => mainWindow.webContents.send('ai-stream-chunk', 'const '), 5
```

```
      setTimeout(() => mainWindow.webContents.send('ai-stream-chunk', 'x = '), 1000
      setTimeout(() => mainWindow.webContents.send('ai-stream-chunk', '42;'), 1500

      return "Stream started";
  });
```

How it works:

- `ipcMain.handle` sets up a listener on the backend. When React calls `window.api.getFileContents`, it triggers the 'get-file-contents' handler here.
- We use the **Invoke/Handle** pattern (Promises) as mandated by your architecture guardrails. This prevents memory leaks.
- For the AI simulation, we use `webContents.send` to push data back to the frontend in chunks, simulating how a real LLM API (like OpenAI) streams tokens.

Step 6: Test the Shell in React (`App.jsx`)

The React UI component that interacts with our secure bridge.

```
import { useState, useEffect } from 'react';

function App() {
  const [stream, setStream] = useState("");

  useEffect(() => {
    // Start listening to the bridge when the component mounts
    window.api.onAIStream((chunk) => {
        setStream((prev) => prev + chunk);
    });
  }, []);

  const handleTestBridge = async () => {
```

```
    // Await the promise returned by the backend
    const result = await window.api.getFileContents('/test/file.js');
    console.log("Backend replied:", result);

    // Trigger the backend to start sending stream chunks
    await window.api.sendAIPrompt("Write a variable");
  };

  return (
```

comπle Code Editor

Phase 1 & 3: JS Shell and Bridge Active.

Test IPC Bridge & AI Stream

{stream}

```
  )
}

export default App
```

How it works:

- We use React's `useEffect` to subscribe to the `onAIStream` event listener we defined in the preload script. Every time a chunk arrives, we append it to the `stream` state variable.
- The `handleTestBridge` function tests our bi-directional communication. It sends a request to the backend, logs the response, and then tells the backend to begin the simulated AI stream.